

```
In [64]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)
```

```
In [65]: print("==== FINAL DATASET VALIDATION =====")

print("Rows:", len(df))
print("Columns:", len(df.columns))

print("\nColumns:")
print(df.columns.tolist())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Order IDs:")
print(df["Order_ID"].duplicated().sum())

print("\nOrder Segments:")
print(df["Order_Segment"].value_counts())

print("\nFulfillment Status:")
print(df["Fulfillment_Status"].value_counts())

print("\nFinal Dataset Preview:")
display(df.head())
```

===== FINAL DATASET VALIDATION =====

Rows: 1000

Columns: 16

Columns:

['Order_ID', 'Customer_ID', 'Category', 'Payment_Method', 'Order_Value',
'Discount_Percent', 'Est_Delivery_Days', 'Actual_Delivery_Days', 'Customer
_Rating', 'Returned', 'City_Tier', 'Delivery_Delay', 'Fulfillment_Status',
'High_Value_Flag', 'Net_Revenue', 'Order_Segment']

Missing Values:

Order_ID	0
Customer_ID	0
Category	0
Payment_Method	0
Order_Value	0
Discount_Percent	0
Est_Delivery_Days	0
Actual_Delivery_Days	0
Customer_Rating	0
Returned	0
City_Tier	0
Delivery_Delay	0
Fulfillment_Status	0
High_Value_Flag	0
Net_Revenue	0
Order_Segment	0

dtype: int64

Duplicate Order IDs:

0

Order Segments:

Order_Segment	
Standard Order	756
Loyal & Satisfied	214
High-Risk Transaction	30

Name: count, dtype: int64

Fulfillment Status:

Fulfillment_Status	
On-Time	450
Minor Delay	288
Early	154
Severe Delay	108

Name: count, dtype: int64

Final Dataset Preview:

	Order_ID	Customer_ID	Category	Payment_Method	Order_Value	Discount_Perc
0	ORD1000	CUST202	Beauty	Credit Card	1167.26	
1	ORD1001	CUST279	Electronics	Cash on Delivery	3381.32	
2	ORD1002	CUST192	Home & Kitchen	Debit Card	416.68	
3	ORD1003	CUST114	Fashion	Debit Card	792.90	
4	ORD1004	CUST206	Beauty	Cash on Delivery	1770.58	

```
In [66]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("ecommerce_orders.csv")

print("DataFrame Shape:")
print(df.shape)

print("\nColumn Data Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nFirst 5 Rows:")
display(df.head())
```

DataFrame Shape:
(1015, 11)

Column Data Types:

```
Order_ID          object
Customer_ID       object
Category          object
Payment_Method    object
Order_Value       float64
Discount_Percent  int64
Est_Delivery_Days int64
Actual_Delivery_Days int64
Customer_Rating   float64
Returned          object
City_Tier         object
dtype: object
```

Missing Values:

```
Order_ID          0
Customer_ID       0
Category          0
Payment_Method    0
Order_Value       20
Discount_Percent  0
Est_Delivery_Days 0
Actual_Delivery_Days 0
Customer_Rating   45
Returned          0
City_Tier         0
dtype: int64
```

First 5 Rows:

	Order_ID	Customer_ID	Category	Payment_Method	Order_Value	Discount_Percent
0	ORD1000	CUST202	Beauty	Credit Card	NaN	
1	ORD1001	CUST279	Electronics	Cash on Delivery	3381.32	
2	ORD1002	CUST192	Home & Kitchen	Debit Card	416.68	
3	ORD1003	CUST114	Fashion	Debit Card	792.90	
4	ORD1004	CUST206	Beauty	Cash on Delivery	1770.58	

```
In [67]: print("Duplicate Order_ID count:")
print(df["Order_ID"].duplicated().sum())

df = df.drop_duplicates(
    subset="Order_ID",
    keep="first"
).copy()

print("\nShape after deduplication:")
print(df.shape)

print("\nDuplicate Order_IDs after cleaning:")
print(df["Order_ID"].duplicated().sum())
```

Duplicate Order_ID count:
15

Shape after deduplication:
(1000, 11)

Duplicate Order_IDs after cleaning:
0

```
In [68]: df["Payment_Method"] = (
          df["Payment_Method"]
          .str.strip()
          .str.lower()
        )

df["Payment_Method"] = df["Payment_Method"].replace({
    "upi": "UPI",
    "credit card": "Credit Card",
    "debit card": "Debit Card",
    "cash on delivery": "Cash on Delivery"
})

print(df["Payment_Method"].value_counts())
```

```
Payment_Method
UPI                389
Cash on Delivery   227
Credit Card       200
Debit Card        184
Name: count, dtype: int64
```

```
In [69]: print("Missing Order_Value before imputation:")
print(df["Order_Value"].isnull().sum())
```

Missing Order_Value before imputation:
20

```
In [70]: category_median = (
          df.groupby("Category")["Order_Value"]
          .transform("median")
        )

df["Order_Value"] = df["Order_Value"].fillna(category_median)
```

```
In [71]: print("Missing Order_Value after imputation:")
print(df["Order_Value"].isnull().sum())
```

Missing Order_Value after imputation:
0

```
In [72]: print("Missing Customer_Rating before:")
print(df["Customer_Rating"].isnull().sum())

rating_mode = df["Customer_Rating"].mode()[0]

df["Customer_Rating"] = df["Customer_Rating"].fillna(
    rating_mode
)

print("\nRating mode used:", rating_mode)
```

```
print("Missing Customer_Rating after:")
print(df["Customer_Rating"].isnull().sum())
```

Missing Customer_Rating before:
44

Rating mode used: 4.0
Missing Customer_Rating after:
0

```
In [73]: est_delivery_days = df["Est_Delivery_Days"].to_numpy()

actual_delivery_days = df["Actual_Delivery_Days"].to_numpy()

delivery_delay = (
    actual_delivery_days - est_delivery_days
)

df["Delivery_Delay"] = delivery_delay

display(
    df[
        [
            "Est_Delivery_Days",
            "Actual_Delivery_Days",
            "Delivery_Delay"
        ]
    ].head(10)
)
```

	Est_Delivery_Days	Actual_Delivery_Days	Delivery_Delay
0	3	3	0
1	6	6	0
2	7	8	1
3	7	6	-1
4	5	7	2
5	7	6	-1
6	4	4	0
7	6	8	2
8	3	3	0
9	7	7	0

```
In [74]: conditions = [
    df["Delivery_Delay"] < 0,
    df["Delivery_Delay"] == 0,
    (df["Delivery_Delay"] > 0) & (df["Delivery_Delay"] <= 2),
    df["Delivery_Delay"] > 2
]

choices = [
    "Early",
```

```

        "On-Time",
        "Minor Delay",
        "Severe Delay"
    ]

    df["Fulfillment_Status"] = np.select(
        conditions,
        choices,
        default="Unknown"
    )

    print(df["Fulfillment_Status"].value_counts())

```

```

Fulfillment_Status
On-Time          450
Minor Delay      288
Early            154
Severe Delay     108
Name: count, dtype: int64

```

```

In [75]: percentile_95 = np.percentile(
        df["Order_Value"],
        95
    )

    print("95th Percentile:", percentile_95)

    df["High_Value_Flag"] = np.where(
        df["Order_Value"] > percentile_95,
        "High-Value Order",
        "Regular Order"
    )

    print("\nHigh-Value Order Count:")
    print(df["High_Value_Flag"].value_counts())

```

```

95th Percentile: 3628.5789999999993

```

```

High-Value Order Count:
High_Value_Flag
Regular Order      950
High-Value Order    50
Name: count, dtype: int64

```

```

In [76]: df["Net_Revenue"] = (
        df["Order_Value"] *
        (1 - df["Discount_Percent"] / 100)
    )

    display(
        df[
            [
                "Order_Value",
                "Discount_Percent",
                "Net_Revenue"
            ]
        ].head(10)
    )

```

	Order_Value	Discount_Percent	Net_Revenue
0	1167.26	20	933.808
1	3381.32	20	2705.056
2	416.68	5	395.846
3	792.90	15	673.965
4	1770.58	0	1770.580
5	2767.12	0	2767.120
6	1533.83	0	1533.830
7	558.37	10	502.533
8	1547.27	10	1392.543
9	622.41	10	560.169

```
In [77]: category_performance = (
    df.groupby("Category")
      .agg(
        Total_Net_Revenue=("Net_Revenue", "sum"),
        Average_Discount=("Discount_Percent", "mean"),
        Return_Rate=(
            "Returned",
            lambda x: (x == "Yes").mean() * 100
        )
      )
    .sort_values(
        "Total_Net_Revenue",
        ascending=False
    )
)

category_performance["Total_Net_Revenue"] = (
    category_performance["Total_Net_Revenue"].round(2)
)

category_performance["Average_Discount"] = (
    category_performance["Average_Discount"].round(2)
)

category_performance["Return_Rate"] = (
    category_performance["Return_Rate"].round(2)
)

display(category_performance)
```

	Total_Net_Revenue	Average_Discount	Return_Rate
Category			
Fashion	339490.35	12.43	18.48
Electronics	323786.39	12.50	21.85
Home & Kitchen	266432.08	12.31	16.35
Beauty	207861.68	13.16	15.19
Books	103016.35	12.10	23.86

```
In [78]: tier_analysis = (
    df.groupby("City_Tier")
      .agg(
        Total_Orders=("Order_ID", "count"),
        Severe_Delays=(
            "Fulfillment_Status",
            lambda x: (x == "Severe Delay").sum()
        )
      )
    )

tier_analysis["Severe_Delay_Rate"] = (
    tier_analysis["Severe_Delays"] /
    tier_analysis["Total_Orders"] *
    100
)

tier_analysis = tier_analysis.sort_values(
    "Severe_Delay_Rate",
    ascending=False
)

display(tier_analysis)
```

	Total_Orders	Severe_Delays	Severe_Delay_Rate
City_Tier			
Tier 2	346	41	11.849711
Tier 1	472	53	11.228814
Tier 3	182	14	7.692308

```
In [79]: worst_tier = tier_analysis["Severe_Delay_Rate"].idxmax()

worst_rate = tier_analysis.loc[
    worst_tier,
    "Severe_Delay_Rate"
]

print(
    f"Worst City Tier: {worst_tier}"
)

print(
```

```
f"Severe Delay Rate: {worst_rate:.2f}%"
)
```

Worst City Tier: Tier 2

Severe Delay Rate: 11.85%

```
In [80]: rating_pivot = pd.pivot_table(
    df,
    values="Customer_Rating",
    index="Category",
    columns="Fulfillment_Status",
    aggfunc="mean"
)

rating_pivot = rating_pivot.round(2)

display(rating_pivot)
```

Fulfillment_Status	Early	Minor Delay	On-Time	Severe Delay
Category				
Beauty	3.24	3.10	3.51	3.48
Books	3.68	2.95	3.35	3.75
Electronics	3.46	3.48	3.48	3.48
Fashion	3.42	3.43	3.32	3.52
Home & Kitchen	3.10	3.55	3.40	3.42

```
In [81]: median_order_value = df["Order_Value"].median()

print(
    "Median Order Value:",
    round(median_order_value, 2)
)
```

Median Order Value: 1099.02

```
In [82]: conditions = [

    (
        (df["Discount_Percent"] >= 30) &
        (df["Returned"] == "Yes")
    ),

    (
        (df["Order_Value"] > median_order_value) &
        (df["Customer_Rating"] >= 4) &
        (df["Returned"] == "No")
    )
]

choices = [
    "High-Risk Transaction",
    "Loyal & Satisfied"
]

df["Order_Segment"] = np.select(
```

```

        conditions,
        choices,
        default="Standard Order"
    )

    print(df["Order_Segment"].value_counts())

```

```

Order_Segment
Standard Order          756
Loyal & Satisfied       214
High-Risk Transaction    30
Name: count, dtype: int64

```

```

In [83]: revenue_by_category = (
    df.groupby("Category")["Net_Revenue"]
        .sum()
        .sort_values(ascending=False)
    )

plt.figure(figsize=(10, 6))

bars = plt.bar(
    revenue_by_category.index,
    revenue_by_category.values
)

plt.title(
    "Total Net Revenue by Product Category",
    fontsize=15
)

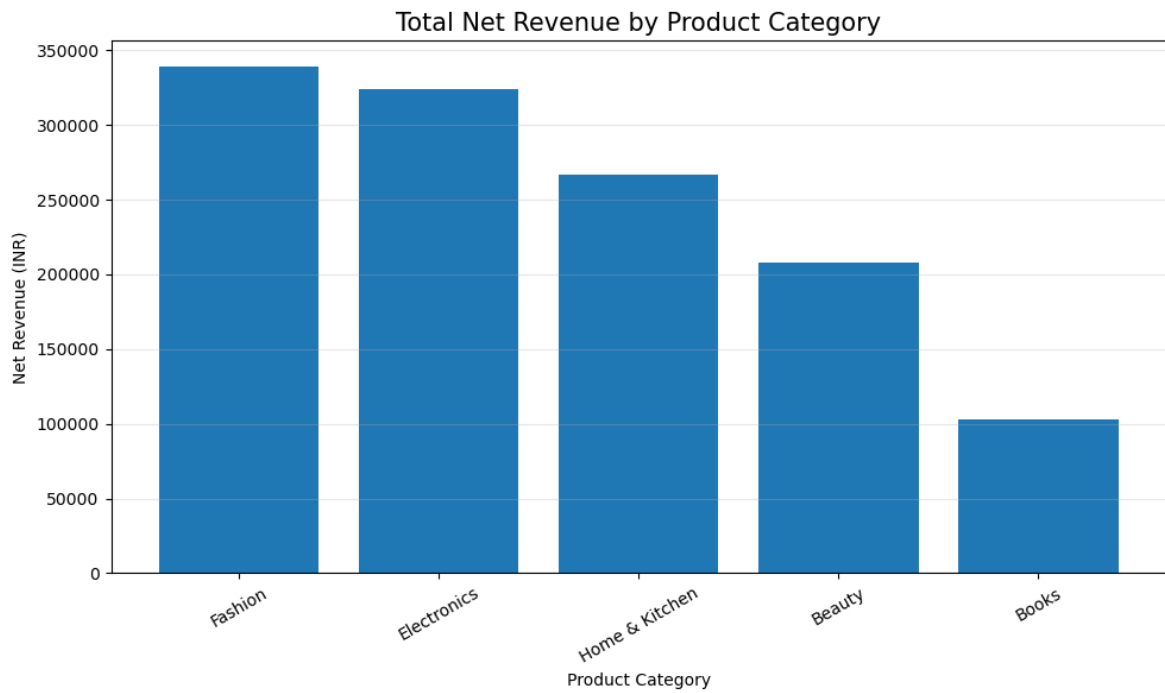
plt.xlabel("Product Category")
plt.ylabel("Net Revenue (INR)")

plt.xticks(rotation=30)

plt.grid(
    axis="y",
    alpha=0.3
)

plt.tight_layout()
plt.show()

```



```
In [84]: plt.figure(figsize=(10, 6))

plt.hist(
    df["Actual_Delivery_Days"],
    bins=range(
        df["Actual_Delivery_Days"].min(),
        df["Actual_Delivery_Days"].max() + 2
    ),
    edgecolor="black"
)

average_estimated_delivery = (
    df["Est_Delivery_Days"].mean()
)

plt.axvline(
    average_estimated_delivery,
    linestyle="--",
    linewidth=2,
    label=(
        f"Average Estimated Delivery = "
        f"{average_estimated_delivery:.2f} days"
    )
)

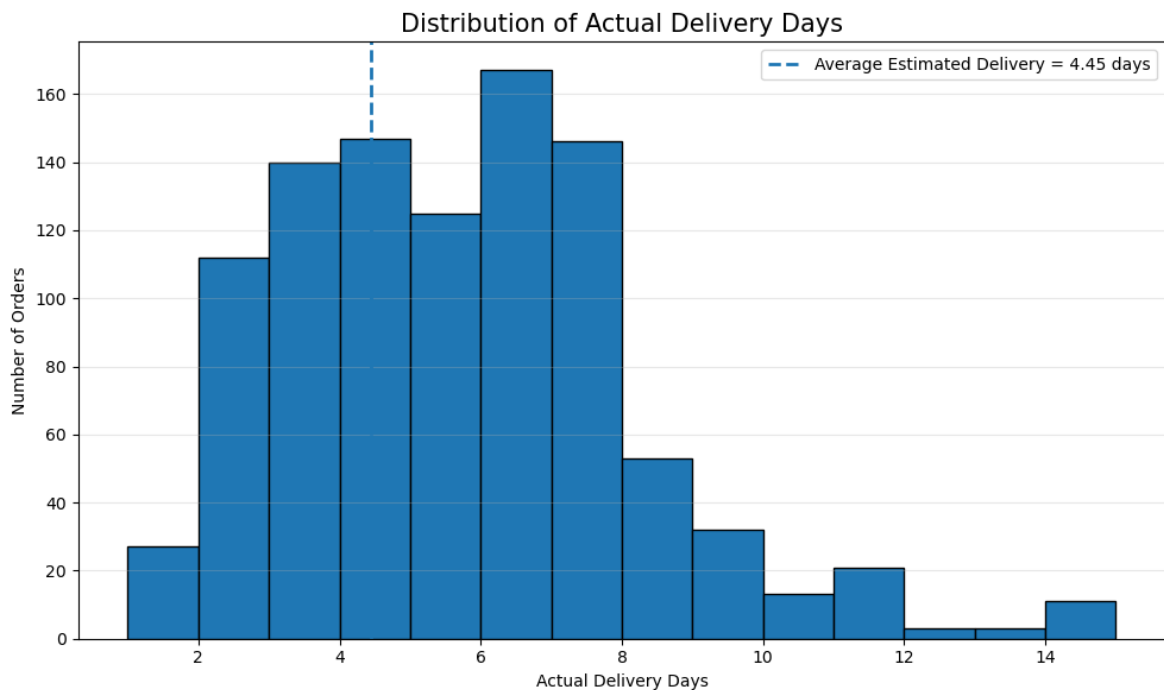
plt.title(
    "Distribution of Actual Delivery Days",
    fontsize=15
)

plt.xlabel("Actual Delivery Days")
plt.ylabel("Number of Orders")

plt.grid(
    axis="y",
    alpha=0.3
)
```

```
plt.legend()

plt.tight_layout()
plt.show()
```



```
In [85]: plt.figure(figsize=(10, 6))

returned_no = df["Returned"] == "No"
returned_yes = df["Returned"] == "Yes"

plt.scatter(
    df.loc[returned_no, "Discount_Percent"],
    df.loc[returned_no, "Order_Value"],
    label="Not Returned",
    alpha=0.6
)

plt.scatter(
    df.loc[returned_yes, "Discount_Percent"],
    df.loc[returned_yes, "Order_Value"],
    label="Returned",
    alpha=0.6
)

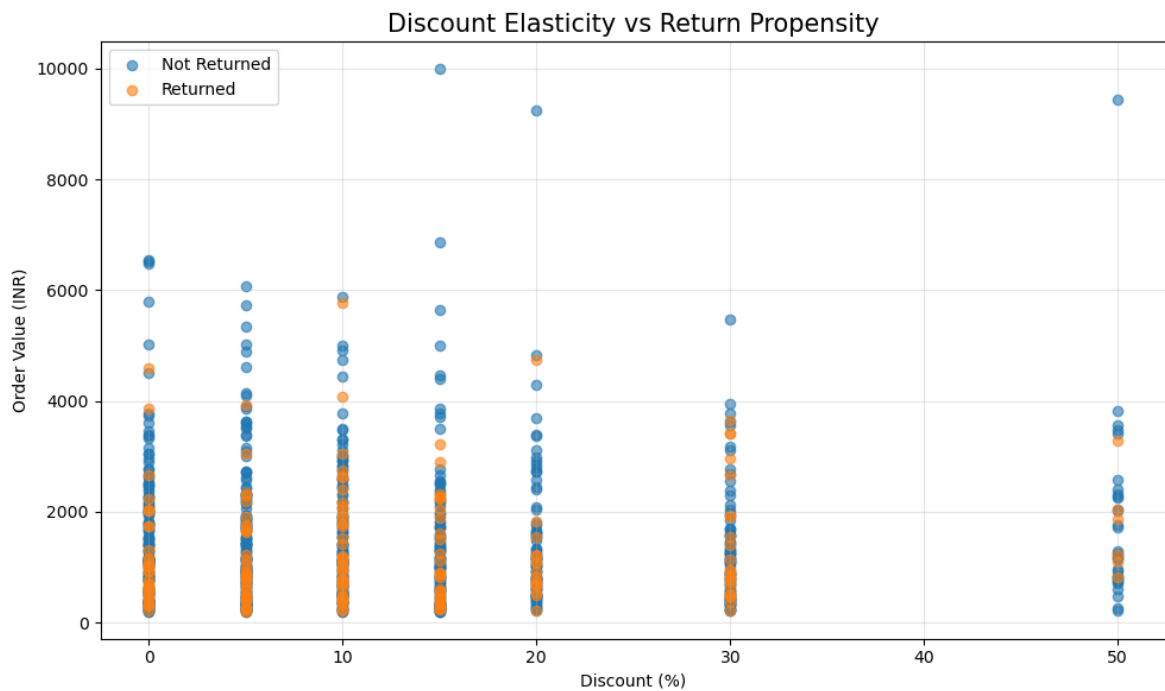
plt.title(
    "Discount Elasticity vs Return Propensity",
    fontsize=15
)

plt.xlabel("Discount (%)")
plt.ylabel("Order Value (INR)")

plt.grid(
    alpha=0.3
)

plt.legend()
```

```
plt.tight_layout()
plt.show()
```



```
In [86]: status_order = [
    "Early",
    "On-Time",
    "Minor Delay",
    "Severe Delay"
]

rating_by_status = (
    df.groupby("Fulfillment_Status")["Customer_Rating"]
    .mean()
    .reindex(status_order)
)

plt.figure(figsize=(10, 6))

plt.bar(
    rating_by_status.index,
    rating_by_status.values
)

plt.title(
    "Logistics Impact on Customer Sentiment",
    fontsize=15
)

plt.xlabel("Fulfillment Status")
plt.ylabel("Average Customer Rating (Stars)")

plt.ylim(0, 5)

plt.grid(
    axis="y",
    alpha=0.3
)
```

```
plt.tight_layout()
plt.show()
```



```
In [88]: df.to_csv("ecommerce_orders_cleaned.csv", index=False)

print("Cleaned dataset saved successfully!")
print("File name: ecommerce_orders_cleaned.csv")
```

Cleaned dataset saved successfully!
File name: ecommerce_orders_cleaned.csv

```
In [92]: !jupyter nbconvert --to html ecommerce.ipynb
```

[NbConvertApp] WARNING | pattern 'ecommerce.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
to various other formats.

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options

=====

The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

<cmd> --help-all

--debug

set log level to logging.DEBUG (maximize logging output)

Equivalent to: [--Application.log_level=10]

--show-config

Show the application's configuration (human-readable format)

Equivalent to: [--Application.show_config=True]

--show-config-json

Show the application's configuration (json format)

Equivalent to: [--Application.show_config_json=True]

--generate-config

generate default config file

Equivalent to: [--JupyterApp.generate_config=True]

-y

Answer yes to any questions instead of prompting.

Equivalent to: [--JupyterApp.answer_yes=True]

--execute

Execute the notebook prior to export.

Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors

Continue notebook execution even if one of the cells throws an error and include the error message in the cell output (the default behaviour is to abort conversion). This flag is only relevant if '--execute' was specified, too.

Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin

read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'

Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout

Write notebook output to stdout instead of files.

Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]

--inplace

Run nbconvert in place, overwriting the existing notebook (only relevant when converting to notebook format)

Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory=]

--clear-output

Clear output of current file and save in place, overwriting the existing notebook.

Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory= --ClearOutputPreprocessor.enabled=True]

--coalesce-streams

Coalesce consecutive stdout and stderr outputs into one stream (within each cell).

Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory= --CoalesceStreamsPre

```

processor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True --TemplateExporter.exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
    This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True --TemplateExporter.exclude_input=True --TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]
--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
    Embed the images as base64 dataurls in the output. This flag is only useful for the HTML/WebPDF/Slides exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICAL']
    Default: 30
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
    ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
    or a dotted object name that represents the import path for an ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed as prebuilt extension for the lab template)

```

```

    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized. This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
                                results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
                                results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
                                Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
                                to output to the directory of each notebook.
    To recover
                                previous default behaviour (outputting to
    o the current
                                working directory) use . as the flag value.
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
                                This defaults to the reveal CDN, but can be any url pointing to
    o a copy
                                of reveal.js.
                                For speaker notes to work, this must be a relative path to a local
    ocal
                                copy of reveal.js: e.g., "reveal.js".
                                If a relative path is given, it must be a subdirectory of the
                                current directory (from which the server is run).
                                See the usage documentation
                                (https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-
                                js-html-slideshow)
                                for more details.
    Default: ''
    Equivalent to: [--SlidesExporter.reveal_url_prefix]
--nbformat=<Enum>
    The nbformat version to write.
                                Use this to downgrade notebooks.
    Choices: any of [1, 2, 3, 4]
    Default: 4
    Equivalent to: [--NotebookExporter.nbformat_version]

```

Examples

The simplest way to use nbconvert is

```
> jupyter nbconvert mynotebook.ipynb --to html
```

Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

```
> jupyter nbconvert --to latex mynotebook.ipynb
```

Both HTML and LaTeX support multiple output templates. LaTeX includes

'base', 'article' and 'report'. HTML includes 'basic', 'lab' and

'classic'. You can specify the flavor of the format used.

```
> jupyter nbconvert --to html --template lab mynotebook.ipynb
```

You can also pipe the output to stdout, rather than a file

```
> jupyter nbconvert mynotebook.ipynb --stdout
```

PDF is generated via latex

```
> jupyter nbconvert mynotebook.ipynb --to pdf
```

You can get (and serve) a Reveal.js-powered slideshow

```
> jupyter nbconvert myslides.ipynb --to slides --post serve
```

Multiple notebooks can be given at the command line in a couple of

different ways:

```
> jupyter nbconvert notebook*.ipynb
```

```
> jupyter nbconvert notebook1.ipynb notebook2.ipynb
```

or you can specify the notebooks list in a config file, containing::

```
c.NbConvertApp.notebooks = ["my_notebook.ipynb"]
```

```
> jupyter nbconvert --config mycfg.py
```

To see all available configurables, use '--help-all'.

In []: